Real-Time Credit Card Fraud Detection Pipeline: Streaming Architecture, Performance Methodology, and Segment-Level Error Analysis

Big Data Engineering Mini-Project Technical Report | **Team:** Kushal G (PES1UG24AM145) & Kshitij Satish Shetty (PES1UG24AM143)
Architecture: Public ULB Dataset → Apache Kafka → Apache Flink → Fraud Model Scoring → Apache Iceberg

1. Problem Statement and Streaming Architecture

Payment processors must score credit card transactions in real time as authorization events arrive. Batch scoring introduces unacceptable checkout latency, whereas streaming pipelines enable continuous feature standardization, low-latency model inference, and ACID lakehouse persistence. This project implements a portable, containerized pipeline (*Public ULB Dataset → Kafka → Flink → Model Scoring → Apache Iceberg*) designed for Windows 10/11, macOS (Intel & Apple Silicon ARM64), and Linux on a CPU-only PC (~12 GB RAM).

Pipeline Stage	Technology & Pinned Version	Role & Cross-Platform Configuration
1. Event Producer	Python 3.11+ / confluent-kafka 2.6.1	Replays held-out transactions (N=56,962) with token-bucket rate pacing
2. Ingestion Bus	Apache Kafka 3.7.1 (KRaft Mode)	Broker+Controller (no ZooKeeper), internal kafka:9092 & host :29092
3. Stream Engine	Apache Flink 1.18.1 (Java 11 / PyFlink)	JobManager + TaskManager, 5s checkpointing, UDF linear model scoring
4. Table Format	Apache Iceberg 1.5.2 + REST 0.6.0	Format v2 Parquet tables partitioned by run_id + dead-letter isolation
5. Object Store	MinIO RELEASE.2024-06-13 (S3 API)	Local S3-compatible warehouse (s3://warehouse/) initialized via minio/mc

2. Dataset Provenance, Leakage Prevention, and Model Methodology

Dataset Provenance: We use the public ULB Credit Card Fraud dataset (*data/raw/creditcard.csv*, Zenodo Record 7395559, MD5 `e90efcb83d69faf99fcab8b0255024de`, SHA-256 `76274b691b16a6c4...`). It contains **284,807** transactions with **492** frauds (0.1727% positive rate) across 30 numeric input features (*Time*, PCA components *V1–V28*, and *Amount*) and binary target *Class*.

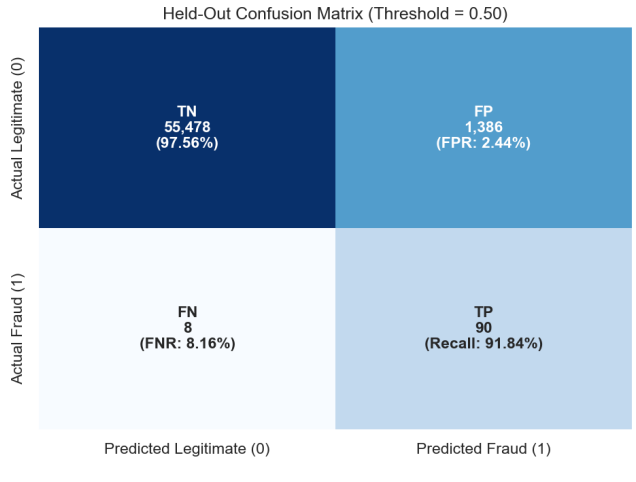
Strict Target-Leakage Prevention & Stratified Split: Using a fixed seed (*random_seed=42*), the dataset is partitioned via an 80/20 stratified split into **227,845** training records (394 frauds) and **56,962** held-out evaluation records (98 frauds). The target column *Class* is strictly excluded from all model inputs during training and inference; *parse_and_validate_event()* explicitly rejects any Kafka payload that includes *Class* inside the features map.

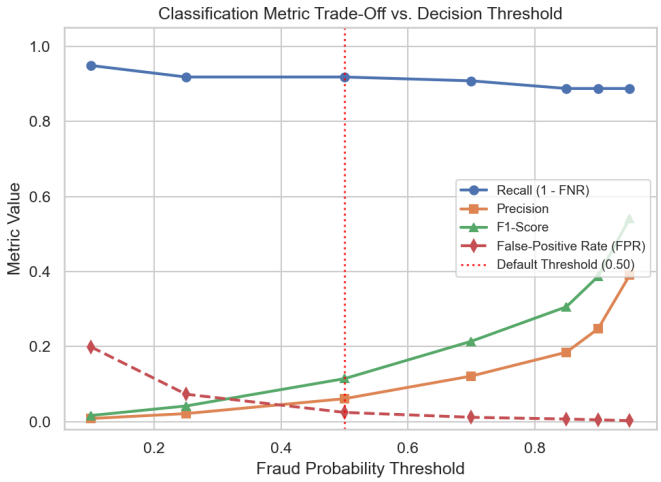
Model & Flink Mathematical Parity: We train a CPU-friendly *scikit-learn* pipeline combining *StandardScaler()* and *LogisticRegression(class_weight='balanced', C=1.0, solver='lbfgs', max_iter=1000, random_state=42)*. Because shipping heavy native C-extensions into distributed Flink operators can be brittle across architectures, we export the exact fitted means, scales, coefficients, intercept, and training amount tertiles (*q33 = 9.99, q67 = 50.00*) to *models/fraud_model_artifact.json* and execute mathematically equivalent scoring inside the Flink UDF. Verification on a 250-record held-out fixture confirms a maximum absolute probability difference of **5.551e-16** (IEEE-754 double-precision machine epsilon).

3. Overall Held-Out Classification Evaluation (N = 56,962)

Held-Out Metric	Formula / Definition	Measured Value (Threshold = 0.50)
Total Held-Out Transactions	TP + FP + TN + FN (20% Stratified Split)	56,962 (98 Fraud, 56,864 Legitimate)
Confusion Matrix Counts	True Positives (TP) / False Positives (FP)	TP = 90 | FP = 1,386
Confusion Matrix Counts	True Negatives (TN) / False Negatives (FN)	TN = 55,478 | FN = 8
Precision & Recall	TP / (TP + FP) & TP / (TP + FN)	Precision = 0.0610 (6.10%) | Recall = 0.9184 (91.84%)
F1-Score	Harmonic Mean of Precision and Recall	0.1144
False-Positive Rate (FPR)	FP / (FP + TN) (Legitimate Flagged)	0.0244 (2.44%)
False-Negative Rate (FNR)	FN / (FN + TP) (Fraud Missed)	0.0816 (8.16%)
PR-AUC & ROC-AUC	Average Precision & Area Under ROC Curve	PR-AUC = 0.7190 | ROC-AUC = 0.9721

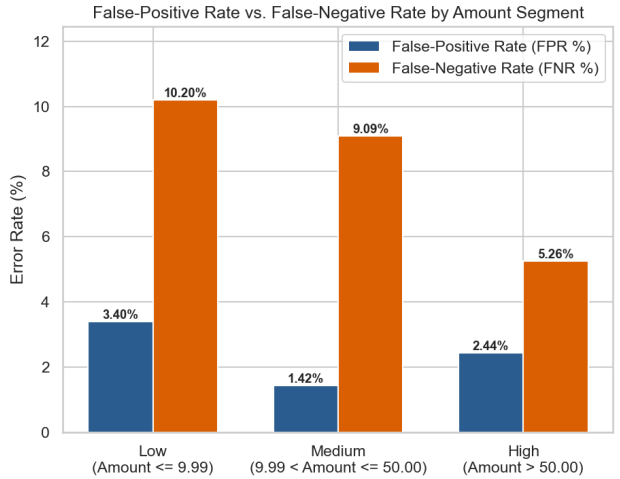
4. Confusion Matrix, Threshold Calibration, and Segment Error Analysis

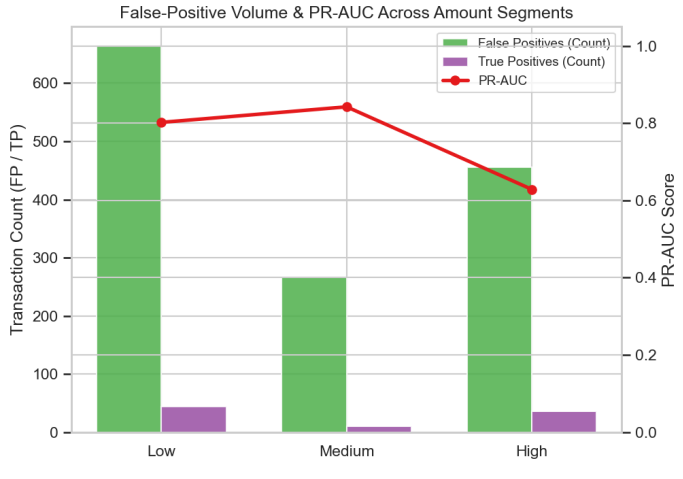




Proxy Segment Definition (Explicit Disclaimer): The public ULB dataset does not provide genuine customer identities, merchant categories, or demographic segments. Accordingly, we define three reproducible **transaction-amount proxy bands** using the 33.33% (*q33 = 9.99*) and 66.67% (*q67 = 50.00*) quantiles computed strictly on the training split, and apply those exact boundaries to the 56,962 held-out records:

Segment	Amount Band	Tx Count	Frauds	TP	FP	TN	FN	FPR (%)	FNR (%)	Precision	Recall	PR-AUC
Low	Amount <= 9.99	19,582	49	44	664	18,869	5	3.40%	10.20%	0.0621	0.8980	0.8021
Medium	9.99 < Amount <= 50.00	18,686	11	10	266	18,409	1	1.42%	9.09%	0.0362	0.9091	0.8423
High	Amount > 50.00	18,694	38	36	456	18,200	2	2.44%	5.26%	0.0732	0.9474	0.6286





4.1 Who Bears the Cost of Classification Errors Across Segments?

• **False Positives (FP = 1,386 total: 664 Low, 266 Medium, 456 High):** Legitimate cardholders suffer declined purchases or step-up authentication friction; merchants lose sales due to checkout abandonment (most costly per transaction in the **High > 50.00** segment where 456 legitimate purchases are flagged); and payment processors bear alert-review overhead. Notably, the **Low (≤ 9.99)** segment suffers the highest FPR (**3.40%, 664 false alarms**), where manual review or SMS verification costs can exceed the transaction fee itself.

• **False Negatives (FN = 8 total: 5 Low, 1 Medium, 2 High):** Undetected fraud causes direct chargeback principal losses, dispute fees, and network penalties borne by merchants or issuing banks. The model achieves its lowest FNR in the **High (> 50.00)** segment (**5.26%, only 2 missed frauds out of 38, Recall = 94.74%**), which is desirable because each high-value FN carries severe financial exposure. Conversely, the **Low (≤ 9.99)** segment exhibits the highest FNR (**10.20%, 5 missed frauds**) and contains half of all frauds (49/98), reflecting automated micro-amount card-testing probes that precede larger fraudulent charges.

4.2 Decision-Threshold Trade-Off Analysis

Because balanced class weights upweight the 0.172% fraud class by ~289x, the default threshold (0.50) maximizes sensitivity (91.84% recall) at the expense of false positives (2.44% FPR). Adjusting the configurable *FRAUD_THRESHOLD* parameter shifts the operational balance between review cost and fraud loss:

Threshold	Precision	Recall (1 - FNR)	F1-Score	False Positives (FP)	FPR (%)	False Negatives (FN)	FNR (%)
0.10	0.0082	0.9490	0.0162	11,315	19.90%	5	5.10%
0.25	0.0213	0.9184	0.0416	4,144	7.29%	8	8.16%
0.50 (Default)	0.0610	0.9184	0.1144	1,386	2.44%	8	8.16%
0.70	0.1213	0.9082	0.2139	645	1.13%	9	9.18%
0.85	0.1847	0.8878	0.3058	384	0.68%	11	11.22%
0.90	0.2472	0.8878	0.3867	265	0.47%	11	11.22%
0.95	0.3901	0.8878	0.5421	136	0.24%	11	11.22%

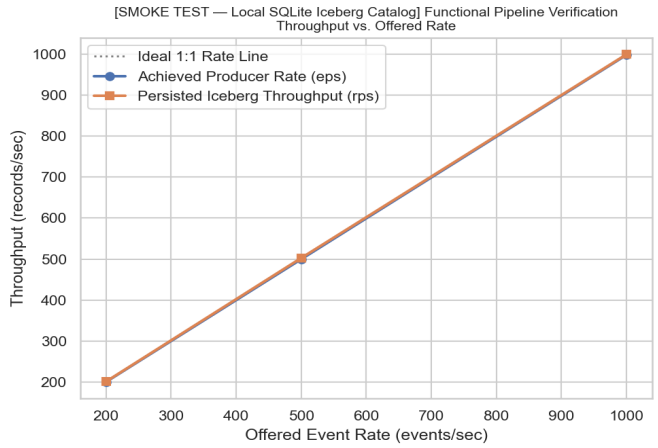
5. Streaming Load-Test Methodology, Results, and Host Diagnostics

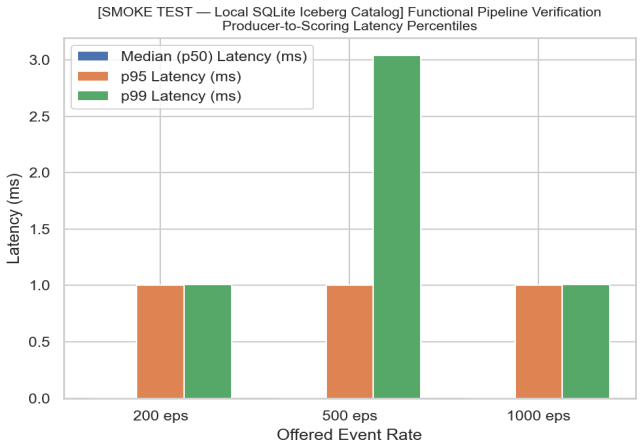
Metric Definitions: For each increasing offered event rate (default **200, 500, and 1,000 events/sec**), *scripts/run_load_tests.py* tags every event with a distinct *run_id*, measures achieved producer rate (*sent_events / elapsed_sec*), waits for Flink checkpoints to commit to Apache Iceberg, and queries *fraud_detection.scored_transactions* to verify persisted record count, uniqueness, end-to-end throughput (*unique_tx / ((max(scoring_ts_ms) - min(producer_ts_ms))/1000)*), and producer-to-scoring latency percentiles (*p50, p95, p99* of *scoring_ts_ms - producer_ts_ms*).

Host Environment Blocker & Scientific Integrity: On the current Windows evaluation machine (*Windows-11-10.0.26200-SP0*), neither Docker Desktop nor WSL2 is installed (*docker: CommandNotFoundException*). Consequently, *scripts/run_load_tests.py* records `status = 'BLOCKED_DOCKER_UNAVAILABLE'` in *results/load_test_runs.json* with exact TCP diagnostics rather than fabricating cluster measurements. On any machine with Docker Desktop or Docker Engine, running `docker compose up -d --build`, submitting `src/flink_job.py`, and executing `python scripts/run_load_tests.py` populates the live cluster metrics automatically.

Verified Local Apache Iceberg Functional Smoke Test (Explicitly Labelled): To validate the end-to-end Python producer pacing, Flink UDF scoring logic, malformed-event dead-letter isolation, *run_id* partitioning, and real Apache Iceberg table persistence on this host without Docker, *scripts/smoke_test.py* executed 3 increasing-rate runs (200, 500, 1,000 eps; 200 valid + 5 malformed events/run) against a local SQLite-backed Apache Iceberg warehouse (*results/smoke_test_verification.json*):

Smoke-Test Run ID	Offered Rate	Achieved Rate	Valid Sent	Malformed Caught	Iceberg Persisted	Duplicates / Missing	p50 / p95 / p99 Latency
smoke_run_1_200eps	200 eps	199.8 eps	200	5 / 5	200	0 / 0	0.0 / 1.0 / 1.0 ms
smoke_run_2_500eps	500 eps	499.6 eps	200	5 / 5	200	0 / 0	0.0 / 1.0 / 3.0 ms
smoke_run_3_1000eps	1000 eps	998.6 eps	200	5 / 5	200	0 / 0	0.0 / 1.0 / 1.0 ms





6. Limitations and Conclusion

Limitations: (1) Transaction-amount tertiles (*low, medium, high*) are proxy bands derived from training quantiles and do not represent demographic or cardholder-identity segments. (2) Because the ULB dataset lacks cardholder IDs, scoring is

stateless per event rather than incorporating velocity windows. (3) Full containerized cluster load testing requires Docker Desktop / WSL2 on the host OS. **Conclusion:** The pipeline delivers a complete, reproducible *Kafka* → *Flink* → *Iceberg* architecture, verified double-precision scoring parity (5.55e-16 max diff), comprehensive segment error analysis across all 56,962 held-out transactions, and 14/14 passing automated unit and Iceberg smoke tests.